

# Schematic of steps to fitting a classification ML model using R H2O – Excluding balancing

## 0. Setup

Load required libraries/tools



## 1. Configuration

Define user-controlled settings:

- Random seed (reproducibility)
- Train/test split ratio
- Evaluation metric (e.g., F1, AUC)
- Cross-validation folds



## 2. Load, inspect and structure data loading

- Import dataset
- Inspect structure and summary statistics
- Identify:
  - Variable types (numeric vs categorical)
  - Target variable
  - Potential issues (missing values, imbalance, etc.)
- Convert categorical variables to appropriate format (e.g., factors)
- Handle problematic variables:
  - Remove IDs
  - Address high-cardinality features
  - Collapse or drop sparse categories
- Re-check data quality



### 3. Define the modelling components

Specify:

- Name of dataset (saved as *df*)
- Name of target variable in the dataset (saved as *target*)



### 4. Train/Test split

- Split data into training and testing sets
- Use stratified sampling (for classification) to preserve class distribution in the test and training sets.
- Run the `set.seed()` function just before the `sample.split()` function to ensure reproducibility.



### 5. Data preprocessing

- *If necessary*, apply transformations (otherwise move onto step 6)
  - **Normalisation/scaling** (for numeric features)
  - **Dummy variable encoding** (for categorical features)
- Fit preprocessing on training data, apply to test data



### 7. Initialize H2O modelling environment

- Start the H2O cluster
- Convert the R training and test set dataframes into H2O dataframes



### 6. Specify the attribute names

Create a list of predictor variables/attributes by removing the target variable from the set of all columns in the training set.



## 8. Hyperparameter tuning (if applicable)

- Define the search space for each hyperparameter (if not necessary move to step 9)
- Grid/random search over all combinations of values specified for the hyperparameters
- Cross-validation-based selection:
  - Order the grid search results according to the best CV performance based on our selected metric
  - Extract the best model ID (according to the first row of the ordered results in the step above)
  - Automatically identify which hyperparameters were tuned
  - Extract the actual tuned values that were used in the best model
  - Append the hyperparameter names to the tuned values



## 9. Training the model

Train the actual model based on the tuned hyperparameter values (if any)



## 10. Generate and extract the predicted probabilities

- Predict probabilities using H2O model for both the training and test sets
- Convert saved predictions:
  - H2O dataframe → R dataframe
- Append the predicted probabilities for the *positive* class only onto the original dataframes for both the training and test sets



## 11. Specify the threshold or find the optimal threshold

Indicate the user-defined threshold to be used to predict the class labels or determine the optimal threshold based on optimising a specific metric.



## 12. Determine the predicted class labels

- For both the training and test sets, convert the predicted probabilities into predicted class labels by assigning the positive class (either a '1' or 'Yes' etc.) if the probability *exceeds* the threshold, otherwise the negative class gets assigned (either a '0' or 'No').
- The resulting predicted class labels are converted to a factor.



## 13. Model evaluation

Using the *predicted class labels* and *actual class labels*, obtain the confusion matrix and model performance on both sets:

- Training set:
  - Confusion matrix
  - ROC curve + AUC
- Test set:
  - Confusion matrix
  - ROC curve + AUC

Use the above to identify the key characteristics of the model, such as bias and variance/overfitting.



## 14. Continue fitting competing models

- Return to *Step 8* to fit additional candidate models and repeat *Steps 9–13* for each model.
- Ensure that all models are trained and evaluated using the same data splits, preprocessing steps, and evaluation metrics for fair comparison.
- Compare model performance across relevant metrics (e.g., Accuracy, Precision, Recall, F1, AUC), taking into account the objective of the problem.
- Assess potential overfitting by comparing training and test performance.
- Consider model interpretability, complexity, and computational efficiency alongside performance.
- Select the best-performing model based on a balance of these factors.

**Additional steps:**

The following additional steps can be integrated along into the pipeline:

- **Balancing:** Balancing of the training set in the case of class imbalance using:
  - Under-sampling
  - Over-sampling
  - Combination of over- and under-sampling
  - SMOTE
  - Ensemble
- **Feature Selection:** Select relevant predictors to reduce noise and improve model performance:
  - Remove irrelevant or redundant variables
  - Methods may include:
    - Filter methods (correlation, statistical tests)
    - Wrapper methods (recursive feature elimination)
    - Embedded methods (e.g., LASSO, tree-based selection)
  - Apply selection using *training data only*, then apply the same selected features to the test set
- **Feature Importance:** Identify which variables contribute most to predictions (model-specific or model-agnostic methods).
- **Model Interpretation:** Explain the direction and impact of key features in the context of the problem (e.g. using SHAP values).